

Dream Machine

Autonomous NSE Trading Bot

Complete System Documentation

Features - Strategies - Process (Auth to Live) - Backtesting Methodology

Generated: 2026-07-17

Zerodha Kite Connect | Python 3.14 | 718 tests

Status: swing-only machine (intraday suspended). 2 validated edges live: donchian_trend_tsl + volatility_contraction_breakout. Paper mode.

1. Executive Summary

Dream Machine is an autonomous intraday+swing trading bot for the NSE (National Stock Exchange of India) built on Zerodha's Kite Connect API. It is a deterministic, rules-based system: the same strategy code runs identically in backtest, paper, and live, and every parameter is validated through a rigorous statistical gauntlet before any capital is committed.

Current state

- The bot is a SWING (daily/positional) machine. The intraday sleeve is SUSPENDED.
- 2 strategies survived the full validation gauntlet and trade live (paper): donchian_trend_tsl (PF 1.72 with ATR trailing) and volatility_contraction_breakout (PF 1.72).
- Across 14 strategies x 4 intraday timeframes with parameter sweeps and slippage, NO intraday edge exists on NSE large-caps - it is structurally a cost sink.
- All money is simulated (paper mode). Going live requires an explicit gated command.

Core design principles

- Pure strategies: candles in, signal out. No side effects; identical bytes across backtest / paper / live.
- Validate before capital: walk-forward out-of-sample, blue-chip confirmation, real delivery costs, plateau, slippage - a strategy must clear all of them.
- Fail safe: circuit breakers, position caps, sector caps, and a hard go-live gate.
- Adaptation only selects pre-validated options or scales by a fixed formula - it never fits or deploys unvalidated parameters live.

2. System Architecture

The bot runs two independent 'sleeves' in one process, each with its own capital book so they never collide on margin.

The two sleeves

Sleeve	Timeframe	Product	Cadence	Status
Intraday	5/15 min	MIS (intraday)	every 5 min, flat by 15:15	SUSPENDED
Swing	Daily	CNC (delivery)	once/day ~15:00, held overn	ACTIVE

The intraday sleeve (scanner + regime router + supertrend/orb) is disabled via config `intraday.enabled=false` because no intraday strategy survived validation. The swing sleeve (SwingEngine) is the live edge.

Component map

- Signals: `strategy.py` (framework + indicators), `strategy_library.py` (19 strategy implementations), `strategy_meta.py` (per-regime parameter sets).
- Regime: `regime.py` (classifier), `market_state.py` (ADX/ATR/BB-width/breadth), `router.py` (weights strategies by fit), `live_router.py` (intraday integration).
- Execution: `auth.py` (Kite session), `order_executor.py` (live MIS orders), `paper_trader.py` (simulated fills), `position_manager.py` (positions + trailing SL), `swing_engine.py` (swing sleeve).
- Risk: `risk_manager.py` (sizing, circuit breakers, sector caps), `go_live.py` (go-live gate).
- Data: `data_fetcher.py` (REST), `data_streamer.py` (WebSocket ticks), `tick_candle_builder.py` (tick->candle), `universe_builder.py` (daily universe), `scanner.py` (ranking).
- Backtest: `backtester.py` (engine), `backtest_runner.py` (multi-TF pipeline), `validation.py` (walk-forward), `auto_tuner.py` (tuner), `costs.py` (cost models), `backtest_store.py` (SQLite).
- Ops: `trade_db.py`, `trade_ledger.py`, `state_store.py`, `alert_manager.py`, `telegram_controller.py`, `logger.py`, `event_calendar.py`, `market_calendar.py`.

3. End-to-End Process: Auth to Live

Step 1 - Authentication (bot.py auth)

The user runs 'bot.py auth' and enters their TOTP (time-based one-time PIN). The bot exchanges it for a Kite access token, saved to token.txt. The assistant/bot can NEVER generate the TOTP - only the user. Credentials live only in config/.env (git-ignored).

Step 2 - Startup (bot.py run)

- Load Kite session from token.txt.
- Apply config overlay: the auto-tuner's validated, bounded parameter set (if present), validated against hard bounds; falls back to config.yaml on any violation.
- Build the dynamic universe: derive ~190 F&O-liquid NSE stocks from the live NFO dump, apply a price band; write data_cache/universe_YYYY-MM-DD.csv. (Intraday feature; inert while intraday is suspended.)
- Seed candles: one REST fetch per symbol seeds history; capped for large universes.
- Subscribe the WebSocket feed (up to 3,000 instruments per connection).
- Initialise: PaperTrader (paper) or OrderExecutor (live), RiskManager, PositionManager, TradeLedger, TradeDB, SwingEngine, alerts, Telegram controller.

Step 3 - The main loop (5-minute cycle)

- Circuit-breaker check (daily loss / trade-count limits).
- Manage open positions: trailing SL, SL/target exits.
- Entry scan: SUSPENDED (intraday.enabled=false). When enabled it would scan the scanner's top-10 via the regime router.
- Heartbeat + state persistence.

Step 4 - The swing sleeve (once/day, ~15:00-15:14)

- Classify the daily regime from the index (NIFTY 50).
- Route: pick the regime-appropriate validated strategies (donchian in trends, vcb in trends+range).
- Scan the watchlist on DAILY candles; sizes by risk %; opens CNC positions.
- Exits managed on the daily bar: donchian rides a 6x ATR trailing stop (bot ratchets it and modifies the GTT); others use a fixed target/stop.
- Positions held overnight; state persisted to JSON across restarts.

Step 5 - Going live (bot.py golive --confirm)

Live trading is gated. The go-live gate (go_live.py) requires: at least 5 trading days of paper history, at least 20 paper trades, profit factor ≥ 1.2 , and positive net P&L. Only then does 'bot.py golive --confirm' flip paper_trading.enabled to false. 'bot.py gopaper' reverts instantly. In live mode real CNC orders + static stop GTTs are placed; the bot modifies the GTT daily as the trailing stop advances (Kite has no native trailing GTT).

4. Strategies

19 strategies are implemented in `strategy_library.py`, mined from a corpus of 15,304 trading videos. Each is a pure function `fn(symbol, df, cfg) -> TradeSignal`. Only 2 survived the full gauntlet. Below is every strategy and its status.

4.1 VALIDATED - live in the swing sleeve

Strategy	Type	OOS PF	Notes
<code>donchian_trend_tsl</code>	Trend breakout	1.72	N-day high/low breakout, 6x ATR trail
<code>volatility_contraction_breakout</code>	Squeeze breakout	1.72	Bollinger-width squeeze then range br

4.2 BENCHED - failed the gauntlet

Strategy	Why benched
<code>bb_mean_reversion</code>	PF 1.09 under delivery cost + slippage (was a 1.20 fragile pass)
<code>double_reversal</code>	Lucky spike: plateau 1/6, blue-chip re-confirm PF 1.06
<code>index_dip_reversion</code>	Blue-chip delivery PF 1.17 (smallcap-inflated on pooled set)
<code>abcd_pattern</code>	Blue-chip delivery PF 1.11
<code>ma_pullback</code>	Net-negative under delivery cost (PF 0.99)
<code>supertrend</code>	Intraday - no edge (OOS PF 1.00), was seeded from one stock
<code>orb_nifty</code>	Intraday - no edge (PF < 1)

4.3 RESEARCH - built + tested, no edge (not wired)

`inside_bar_breakout` (S-23), `fair_value_gap` (S-34), `rsi_reversion` (S-16), `trendline_bounce` (S-18), `engulfing_macd` (S-32), `head_shoulders` (S-20), `dip_buy_momentum` (S-14), `red_to_green_vwap` (S-09), `vwap_break_short` (S-10), `vwap_squeeze` (S-11). All failed walk-forward and/or slippage. `trendline_bounce` was tempting (positive on 72% of names) but failed blue-chip delivery (PF 1.14).

4.4 The two live edges in detail

`donchian_trend_tsl` (S-24)

- Entry: go long on a new N-day high close (default lookback 150), short on a new N-day low.
- Exit: a 6x ATR trailing stop that ratchets each day (let winners run). Target is far - the trailing stop is the real exit.
- Regimes: enabled in strong-trend regimes; disabled in range/quiet.
- Validation: walk-forward OOS PF 1.72 on blue-chips WITH the trailing stop, 9/9 param plateau, survives 0.10% slippage (1.54).

`volatility_contraction_breakout` (S-21)

- Entry: when normalized Bollinger-band width is in its narrowest percentile (default 15th) over the lookback (coiled), trade the break of the recent range.
- Exit: fixed stop at the opposite boundary, R-multiple target.
- Regimes: enabled in trend + range (where coils expand); disabled in quiet.
- Validation: blue-chip delivery OOS PF 1.72, plateau 5/6, positive across names.

5. The Regime Router

The router adapts which strategies trade to the current market regime. It never invents parameters - it only selects among pre-validated per-regime parameter sets and weights them.

How it works

- MarketState: compute ADX, ATR%, Bollinger-band width, and breadth from the index.
- Classify: map to one of 6 regimes (STRONG_TREND_UP/DOWN, RANGE, HIGH_VOL_CHOP, QUIET, UNKNOWN) with hysteresis (a new regime must persist several bars - anti-whipsaw).
- Route: for each strategy, look up its validated parameter set + fit (profit factor, trade count) for this regime; weight by fit x confidence, subject to a fail-safe ceiling.
- Trade-nothing: if no strategy has a positive validated fit in the current regime, the router sits out.

Live swing routing (current)

Regime	Active validated strategies
STRONG_TREND_UP / DOWN	donchian_trend_tsl, volatility_contraction_breakout
RANGE	volatility_contraction_breakout
QUIET	(none - sits out)

The learned map (regime_analyst.py) is designed to replace the seeded fits with real per-regime statistics once the paper ledger accrues enough trades.

6. Backtesting Methodology (The Gauntlet)

This is how a strategy earns the right to trade. It is a stacked series of gates; a strategy must clear ALL of them. This process has repeatedly killed strategies that looked profitable but were not.

Gate 1 - Event-driven bar replay

The Backtester replays historical candles one bar at a time, feeding each expanding window to the SAME `generate_signal()` the live bot uses. Entries fill, SL/target are checked against each bar's high/low (SL assumed first if both hit - conservative), costs deducted. Identical code path to live, so what backtests is what trades.

Gate 2 - Walk-forward, out-of-sample

Not a plain in-sample backtest. Each symbol's history is cut into folds+1 time chunks. For each fold: sweep the parameter grid IN-SAMPLE on the earlier chunk, pick the most stable winner, then test those exact parameters on the NEXT unseen chunk (out-of-sample). All OOS results are stitched together. Parameters that only win on the data they were fitted to collapse here.

Gate 3 - Stability

A parameter set that wins across folds (e.g. 3/3) is evidence; one that wins a single fold is noise.

Gate 4 - Blue-chip confirmation

Re-run on liquid blue-chips only (RELIANCE/TCS/INFY/HCLTECH). This strips out small-cap mega-trend luck. It has caught multiple false positives (e.g. a strategy that scored 1.47 pooled collapsed to 1.05 on blue-chips).

Gate 5 - Correct transaction costs

Net of DELIVERY (CNC) costs for swing - not the cheaper intraday costs. Getting this wrong killed 3 'validated' edges (see Section 7).

Gate 6 - Plateau

For every parameter combo in the grid, run a fixed-param backtest and report how many clear the bar. A robust edge sits on a plateau (most neighbours also work); a fragile one is a lone spike (e.g. 1 of 6 combos).

Gate 7 - Slippage stress

Re-run under execution slippage (0.10%, a hard 2x stress) applied to both entry and exit fills. Fragile edges evaporate; the intraday strategies dropped to PF 0.16-0.47 here.

Gate 8 - Multiple symbols

Confirm the edge generalizes across ~25-30 names (daily bars, some resampled from intraday) - not just the 8 deep-history stocks.

Acceptance bar

Minimum to pass: ≥ 30 out-of-sample trades, profit factor ≥ 1.2 , net P&L > 0 - AND clear the plateau + slippage + blue-chip gates. Still-pending gates (roadmap): 2x-slippage on more names, Monte Carlo trade shuffling, and ≥ 2 weeks of live paper vs backtest divergence.

7. Transaction Cost Model

Costs are subtracted from every trade's P&L in backtest, paper, and live. The model (costs.py) dispatches on product: delivery (swing) or intraday. Getting this right is critical - the delivery model is ~3x costlier and killed 3 edges.

Per Rs.1,00,000 buy + Rs.1,00,000 sell round trip (Zerodha equity)

Charge	Delivery (CNC) - swing	Intraday (MIS)
Brokerage	Rs.0 (free)	Rs.40 (Rs.20/order)
STT	Rs.200 (0.1% both legs)	Rs.25 (0.025% sell)
Exchange txn (NSE)	Rs.5.94	Rs.5.94
SEBI	Rs.0.20	Rs.0.20
Stamp duty	Rs.15 (0.015% buy)	Rs.3
DP charge	Rs.15.34 (per sell)	-
GST (18%)	Rs.3.87	Rs.8.30
TOTAL	~Rs.240 (~0.24%)	~Rs.82 (~0.08%)

For swing (delivery), STT (0.2% round-trip) dominates and there is no brokerage. This is why high-turnover strategies are especially cost-sensitive and several failed once costs were corrected.

8. Risk Management

Position sizing

$qty = \text{floor}(\text{risk_pct} \times \text{capital} / \text{stop_distance})$, then shrunk to fit the tighter of max-position-value and a per-order value cap. A high-priced stock simply takes a smaller size (never skipped).

Circuit breakers (per day)

- Max daily loss: halts new entries when the day's loss exceeds the limit.
- Max trades per day: halts after N trades.
- When halted, exits keep working (as a GTT would) but no new entries and trailing stops freeze.

Other guards

- Sector cap: limits exposure concentration in one sector.
- Spread guard: blocks entry when the bid-ask spread is too wide (liquidity).
- Event-day skip: no entries on configured market-wide or per-stock event days.
- Swing sleeve: separate capital book (default Rs.5,00,000), 1% risk/position, max 5 positions, max 20% of capital per name.

Go-live gate

A hard, code-enforced gate (go_live.py): ≥ 5 days, ≥ 20 paper trades, $PF \geq 1.2$, $net > 0$. Live requires 'bot.py golive --confirm'. Reversible via 'bot.py gopaper'.

9. Universe & Scanner (Intraday - currently inert)

These power the intraday sleeve's stock selection. They still function but are inert while intraday is suspended; the swing sleeve trades a fixed watchlist instead.

Tier-1: Universe builder

Derives the tradeable universe from the live NFO instrument dump: keeps NSE stocks that have F&O contracts (the ~190 institutionally-liquid, MIS-friendly names), applies a price band. Auto-derived daily so it stays current; avoids illiquid small-caps that blow up strategies.

Tier-2: Scanner

Ranks the universe each cycle by a score = $|\text{pct_change}| \times 1.0 + \text{extreme_proximity} \times 2.0 + |\text{gap}| \times 0.5 + \text{relative_volumex} \times 0.4$, and takes the top-N (default 10). The top-10 rotate as intraday momentum shifts.

10. Data Infrastructure

- data_fetcher.py - REST candles and quotes from Kite (with an offline mode for backtests).
- data_streamer.py - Kite WebSocket tick feed (KiteTicker).
- tick_candle_builder.py - builds live candles from ticks (zero per-cycle REST); seeded once at startup, then kept current by the stream.
- candle_cache.py - candle caching layer.
- backtest_store.py - SQLite store of historical candles (1.1 GB): ~63 large-caps at 1-year intraday (1/5/15/30/60 min) and ~10 stocks at ~10-year daily.
- historical_loader.py - loads/refreshes the backtest store from Kite (delta-loading).
- market_calendar.py / event_calendar.py - trading days and event-day skips.
- timeframe.py, mtf_replay.py - timeframe utilities and multi-timeframe replay.

11. Config Overlay & Auto-Tuner

A deterministic, bounded parameter channel - NOT machine learning. It lets the system re-tune indicator parameters within hard limits, but it can never push the bot outside the bounds defined in code.

Auto-tuner (bot.py tune)

- Runs a WALK-FORWARD parameter sweep per strategy (never plain in-sample).
- Picks the most stable fold-winning parameters.
- If the stitched out-of-sample result clears the acceptance bar, writes the winner to config/overlay.yaml.

Overlay safety

- Every field is validated against hard bounds (allowed strategies, min/max stop, min/max target) at startup.
- An invalid overlay is rejected and the bot falls back to config.yaml, with an alert.
- adaptive_bounds.py / adaptive_params.py define the sandbox limits.

Note: all AI/LLM code was removed earlier - the system is fully deterministic. 'overlay' is the renamed, LLM-free parameter channel.

12. Web Application (Cross-device Dashboard)

A React + Vite PWA (frontend/) served by a FastAPI backend (webapp/), reachable from phone and laptop over Tailscale. Auth is TOTP-based (no API key in the browser).

Screens (frontend/src/components)

- Dashboard - status, mode, market state, go-live gate progress.
- Positions - open positions, daily P&L, trade count.
- Strategies - sleeve status (intraday SUSPENDED), validated vs benched swing edges with profit factors, and the research registry.
- Signals - taken and rejected-with-reason signals.
- Equity chart, Backtest runner, Logs, Settings (config editor), Controls (start/stop/pause/square-off), Login, Gate checks.

Backend API (webapp/routers)

- monitor.py - /api/status, /positions, /trades, /signals, /equity.
- strategies.py - /api/strategies (registered, active, allowed, intraday_enabled, swing edges), set-active.
- control.py - start/stop/pause/square-off. backtest.py - run/track backtests. config.py - view/edit config. logs.py - log tail. auth_login.py - TOTP login.

13. Alerts & Remote Control

- alert_manager.py - sends alerts (entries, exits, SL hits, heartbeats, circuit breakers) to Telegram, tagged [PAPER] or [LIVE].
- telegram_controller.py - listens for /stop and /status commands from Telegram.
- command_channel.py - internal command bus between the web app / Telegram and the bot loop.
- Integrations (credentials in .env): Telegram bot, GitHub, Jira.

14. Full Module Inventory (Used / Built-not-used)

Every module in src/, with whether it is on the live path today.

Module	Purpose	In use?
strategy / strategy_library / strategy	Signals + 19 strategies + regime sets	Yes
regime / market_state / router	Regime classify + route	Yes (swing)
live_router	Intraday router integration	Idle (intraday off)
swing_engine	Swing sleeve	Yes
backtester / backtest_runner / validat	Backtest engine + walk-forward	Yes (research)
auto_tuner / param_sweep	Walk-forward tuner	Yes (bot tune)
overlay / adaptive_bounds / adaptive_p	Bounded param channel	Yes (optional)
costs	Delivery + intraday cost models	Yes
risk_manager / position_manager	Sizing, breakers, positions	Yes
auth / order_executor / paper_trader	Session + order execution	Yes
go_live	Go-live gate	Yes
data_fetcher / data_streamer / tick_ca	Market data	Yes
candle_cache / backtest_store / histor	Data storage/loading	Yes
universe_builder / scanner / stock_sel	Stock selection	Idle (intraday off)
volume_profile	Volume profile (S-28/29/31)	Built, unused
mtf_replay / timeframe	Multi-timeframe utils	Partial
regime_analyst	Learned per-regime map from ledger	Scaffold (weekly)
router_backtest / drift_audit	Router backtest, drift detection	Research
trade_db / trade_ledger / state_store	Persistence	Yes
alert_manager / telegram_controller /	Alerts + control	Yes
event_calendar / market_calendar	Event + trading calendars	Yes
ops / logger / drift_audit	Ops + logging	Yes

15. Command-Line Reference (bot.py)

Command	What it does
bot.py auth	Authenticate with Kite (enter TOTP), save token
bot.py run	Start the bot (paper or live per config)
bot.py backtest [args]	Multi-strategy multi-timeframe backtest
bot.py validate	Validation gauntlet on a strategy
bot.py sweep	Parameter sweep
bot.py tune	Walk-forward auto-tuner (writes overlay if a winner passes)
bot.py status	Show current mode, gate progress, paper stats
bot.py golive --confirm	Flip to live (gated)
bot.py gopaper	Revert to paper

16. Testing & Quality

- 718 automated tests (pytest) cover strategies, regime router, backtester, risk, costs, swing engine, web API, and the data layer.
- Deterministic: strategies are pure functions, so tests pin exact behaviour.
- The backtester mirrors live rules (entry fills, trailing SL, square-off, circuit breakers) so backtest results transfer to paper/live.
- The whole suite runs in ~15-25 seconds.

17. Known Limitations & Roadmap

Honest limitations

- Backtest name-universe is small for daily (~8-10 deep-history stocks; ~25 via resampling).
- The exit model in older backtests used fixed stops; the ATR-trailing simulation now matches live donchian, but was only recently corrected.
- Pending gauntlet gates: Monte Carlo, multi-name 2x slippage, and ≥ 2 weeks of live paper-vs-backtest divergence.
- Live swing order execution (CNC entry + GTT stop + daily GTT modify) is scaffolded but not yet built - paper simulates it.

Data-blocked strategies (need external feeds)

- Small-cap momentum family (bull flag, gap-and-go, micro-pullback, short squeeze) and the scanner cohorts need float, catalyst, pre-market, and short-interest data - NOT available from Kite. They can run on the liquid universe but without small-cap selection.
- Sector rotation (S-25) needs sector-index history (available from Kite) plus a cross-sectional backtester - a separate build.

Key learnings

- Intraday on NSE large-caps has no edge on any timeframe - it is a cost sink.
- Validate under the CORRECT product's costs; delivery costs killed 3 'validated' edges.
- Plateau + slippage + blue-chip confirmation each caught false positives single-PF hid.
- The edge is in the holding period (daily/swing), not the strategy cleverness.

Appendix A - File & Directory Structure

The project root is trading-bot/. Large/generated dirs (.venv, node_modules, __pycache__, .git) are omitted. Module-level detail is in Section 14.

Top-level layout

trading-bot/	
bot.py	Unified CLI (auth/run/backtest/validate/sweep/tune/status/golive/gopaper) - proxies the scripts below
main.py	The live bot loop (intraday + swing orchestration)
config/	
config.yaml	All runtime settings + feature flags
.env	Secrets: Kite key/secret, Telegram (git-ignored)
overlay.yaml	Auto-tuner's validated bounded params
events.yaml	Event days to skip
src/	50 engine modules (full inventory in Section 14)
strategies/	9 per-strategy YAMLS (regime param sets + fit)
webapp/	FastAPI backend + built PWA bundle
routers/	8: monitor, strategies, control, backtest, config, logs, auth_login
static/	Compiled PWA served to the browser
server.py, auth.py, stores.py, settings.py, supervisor.py, ...	
frontend/	React + Vite source (rebuilds webapp/static)
src/components/	12: Dashboard, Positions, Strategies, Signals, EquityChart, Backtest, Logs, Settings, Controls, ...
package.json, vite.config.ts	
tests/	61 files, 718 pytest tests
data_cache/	SQLite backtest store (~1.1 GB) + daily universe CSVs
logs/	Trade DBs, swing_state.json, run logs
docs/	This documentation PDF
requirements.txt	Python dependencies
start_bot.bat	Windows launcher (status -> TOTP -> run)
start_webapp.bat	Web app launcher
setup_autostart.ps1	Scheduled-task setup
token.txt	Kite access token (git-ignored, per-day)

Root helper scripts (thin CLI entry points that bot.py proxies)

- auth.py - Kite TOTP authentication.
- backtest.py / backtest_run.py - backtest entry points (multi-TF pipeline).
- sweep.py / validate.py / autotune.py - parameter sweep, validation, auto-tuner.
- build_universe.py - pre-market universe builder (intraday).
- go_live_check.py - go-live gate check. backfill_profiles.py - RVOL profiles.

Git-ignored (never committed)

config/.env, config/overlay.yaml, config/webapp_token.txt, token.txt, data_cache/, logs/, webapp/static/ (built), frontend/node_modules/, cookies.txt. Credentials live only in config/.env.